

UFUG 2602

Data Structure and Algorithm Design:

Course Information

Beiyan Cao, Tengfei Chang

IoT Thrusts and CES

Hong Kong University of Science and Technology (Guangzhou)

UFUG2602: Course info (1)

- **Instructor:** Tengfei Chang (tengfeichang@hkust-gz.edu.cn)
- **Teaching Assistant:**
 - Ningcong Jiang (njiang300@connect.hkust-gz.edu.cn)
 - Manjiang Cao (mcao999@connect.hkust-gz.edu.cn)
- **Class Schedules:**
 - Lecture: Tue, Thu 9:00AM - 10:20PM, Rm 147, E1
 - Lab: Fri 5.00pm - 5.50pm, Rm 227, E1 | Von Neumann Lab (W2-224D)
 - **Office Hours:** Tuesday 11:00 – 12:00 am., W2-606

UFUG2602: Course info (2)

- Textbook:

Data Structures & Algorithms Using Python

By Rance D. Necaie

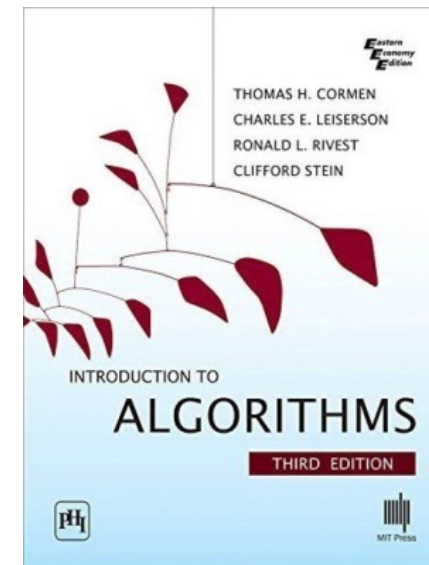
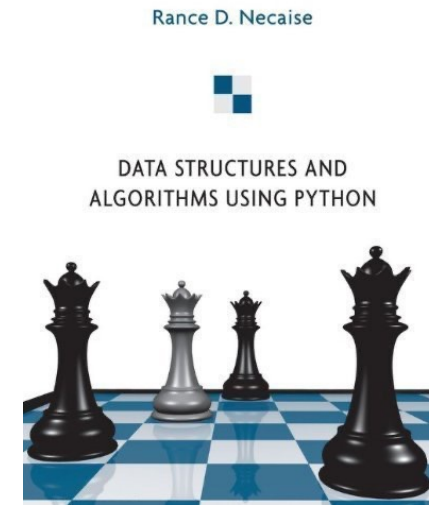
An E-book is in Canvas

- More references:

Introduction to Algorithms

By Thomas H. Cormen, Charles E. Leiserson,

Ronald L. Rivest, and Clifford Stein.



Overview

Date	Course
Week 1	Course Intro, ADT and Arrays
Week 2	Asymptotic Complexity and Search
Week 3	Basic Sorting
Week 4	Lists, Stacks and Queues
Week 5	Recursion
Week 6	Advanced Sorting
Week 7	Hashing
Week 8	Trees Basics and Midterm Exam
Week 9	Tree Traversal
Week 10	Binary Heaps
Week 11	Dynamic Binary Search Tree
Week 12	Huffman Coding
Week 13	Strings (KMP Algorithm)
TBD	Final Exam

Assessment

	Grade	Due Date
In-class Test (Lab)	10%	In-class quiz (TBD)
Mid-term Exam	35%	TBD
Written Assignment (programming)	20%	TBD
Final Exam	35%	TBD

UFUG2602: Intended Learning Outcomes (ILOs)

1. Implement basic data structures and algorithms in Python
2. Solve problems using data structures such as lists, matrices, hash tables, and trees.
3. Design algorithms using methods such as greedy, divide and conquer, and backtrack search.
4. Choose and combine suitable data structures and algorithms for a given problem.

Survey

- Learned data structures or algorithms before?
- Learned programming before joining HKUST(GZ)?
- Anything bothering you in your programming/experience/projects?
- What do you want to learn in this course?

Recommended Learning Style

- **Goal:** Encourage deeper and more diverse understanding via questioning, discussion, and teaching
 - Class participation
 - Exercises (Lab exercises and paper exercises)
 - Review contents timely (using, e.g., Feynman's method)
 - Ask questions
- **Tips:**
 - Learning by doing
 - Good time management skills
 - Ask (yourself/others) "why" and don't settle with one answer (even if it comes from an authority)

UFUG 2602

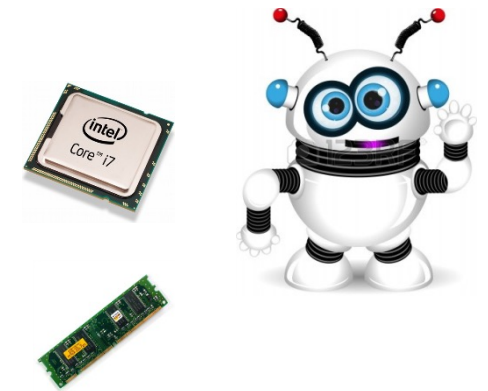
Data Structures and Algorithm Design

Outline

- Introduction
 - The RAM Model
- Data Types
 - User Defined Data Types
 - Abstract Data Types
- Array-based sequences
 - Python Sequence Types
 - Low-level arrays
 - Dynamic Arrays
 - Multidimensional Arrays

Programming Languages?

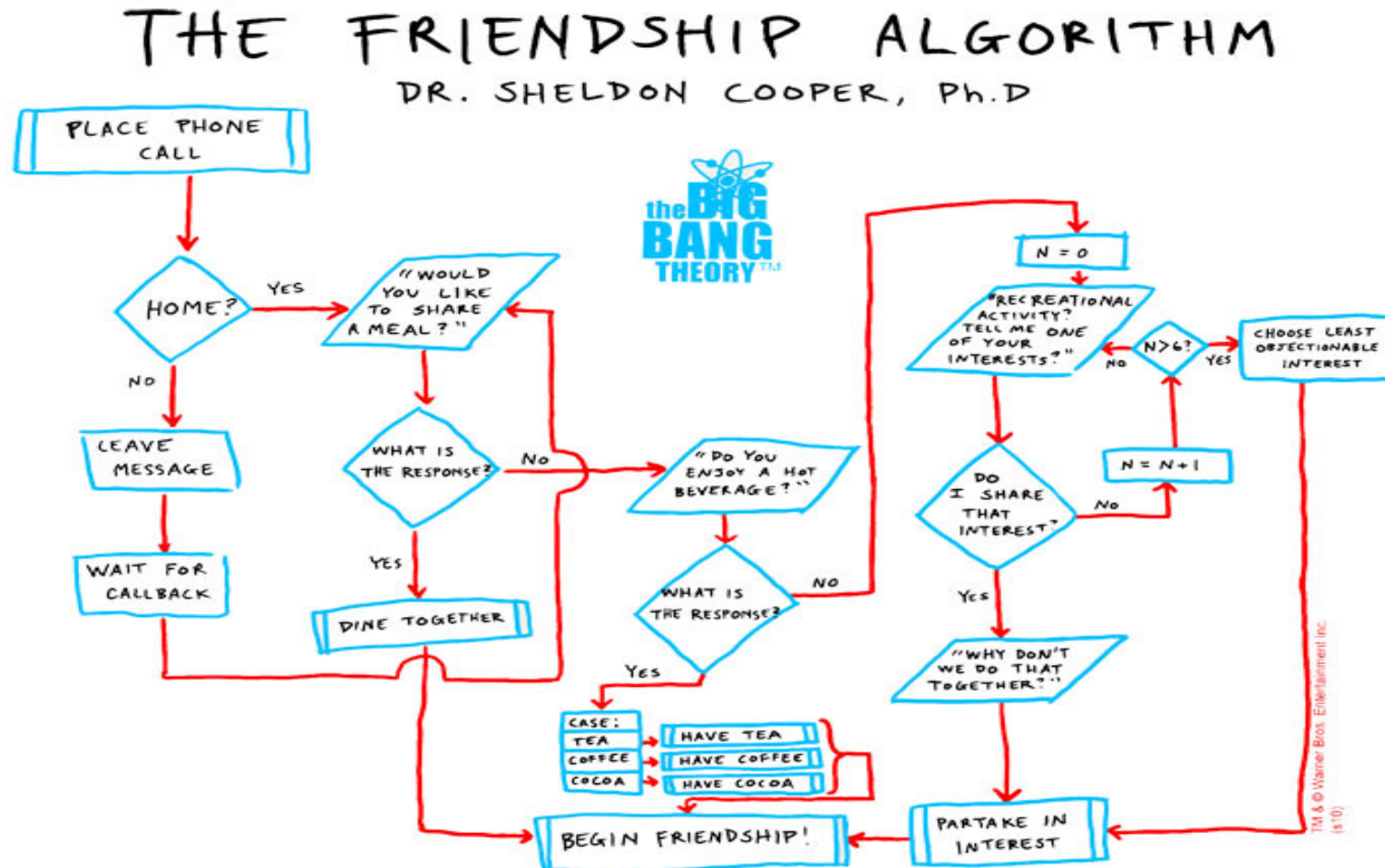
- Natural Languages
 - Chinese, English, Japanese, ...
- Programming Languages (PLs)
 - High Level PLs
 - Pascal, C, Java, ...
 - Low Level PLs
 - Assembly Languages
 - A Machine Language executed by a CPU
- Programming: to tell what CPU to do step-by-step
 - $2 + 3 = ?$
 - Input/Output,
 - Get the answer (computing)



Data Structures and Algorithms

- **Algorithm:** Outline, the essence of a computational procedure, step-by-step instructions
- **Program:** an implementation of an algorithm in some programming language
- **Data structure:** Organization of data needed to solve the problem

What is Algorithm?



An Algorithm is Like a Recipe?

- The ingredients
- The equipment
- The list of steps



Algorithm

- An algorithm is a **finite set of instructions** that, if followed, accomplishes a particular task to solve a problem.
- All algorithms satisfy the following criteria:
 - **Input:** 0 or more quantities are supplied.
 - **Output:** At least one quantity is produced.
 - **Definiteness:** Each instruction is clear and unambiguous.
 - **Finiteness:** For all cases, the algorithm terminates after a finite number of steps.
 - **Effectiveness:** Every instruction must be basic enough (feasible).

Basic Instructions?

- Unlike human, one instruction can only do a very basic thing,
 - read/write a single data value
 - compare two data values
 - Arithmetic operations (+, -, $\times$, $\div$), etc.
 - Depends on computer, programming language, ...
- Consider sorting cards.
 - Human can quickly sort cards in order, because they can see all the cards simultaneously.
 - An algorithm cannot.



Models of Computation

- **Real Computers are complicated**

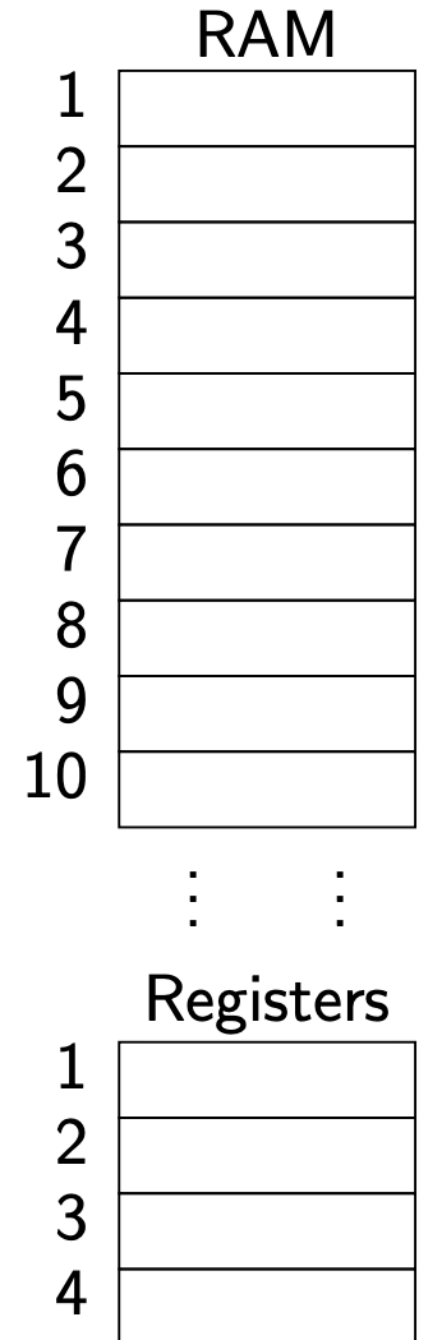
- Memory hierarchy, floating point operations, garbage collector, compiler optimizations, different programming languages, . . .

- **Models of Computation:**

- Simple abstraction of a Computer
- Defines the “Rules of the Game”:
 - Which operations is an algorithm allowed to do?
 - What is the cost of each operation?
 - Cost of an algorithm = $\sum$ cost of all its operations

RAM Model (1)

- **RAM Model:** Random Access Machine Model
 - Infinite Random Access Memory (an array), each cell has a unique address
 - Each cell stores one word, e.g., an integer, a character, an address, etc.
 - **Input:** Stored in RAM
 - **Output:** To be written into RAM
 - A finite (constant) number of registers (e.g., 4)
- **In a single Step we can:**
 - Load a word from memory into a register
 - Compute (+, −, *, /), bit operations, comparisons, etc. on registers
 - Move a word from register to memory



RAM Model (2)

- **Algorithm in the RAM Model**

Sequence of elementary operations (similar to assembler code)

- **Example:** Compute the sum of two integers

Assume that $M[1]$ and $M[2]$ contain the integers

Write output to position $M[3]$

- **Cost of an Algorithm:**

Runtime: Total number of elementary operations

Space: Total number of memory cells used

Take-away Messages

- This is not a programming course; however, you will engage in exercises that focus on problem-solving through programming.
- Programming is the last step in software development, which occurs only after the algorithms are clear.
- Computer science is a branch of mathematics with its art reflected in the beauty of algorithms.
 - Programming knowledge is not necessary to study algorithms.

Outline

- Introduction
 - The RAM Model
- **Data Types**
 - User Defined Data Types
 - Abstract Data Types
- Array-based sequences
 - Python Sequence Types
 - Low-level arrays
 - Dynamic Arrays
 - Multidimensional Arrays

Data Types

- Algorithms are to manipulate data.
- A piece of datum represents something in the real world, such as a student number (integer), loan balance (real number), etc.
- A **data type** is a notion used in programming languages, and is defined as a **collection of objects** (data values) and a **set of operations** that act on those objects.
- For example, the integer data type
 - Values: -100, 0, 200, ...
 - Operations: +, -, / (division), * (multiplication), etc.
- Programming languages provide basic data types such as integer, char, float, double, etc.

Data Structure

- A data structure is a **logical organization of data**.
 - For a student record, the component elements are the fields. These fields describe different attributes of a student.
 - In Python, dictionaries and classes are commonly used to group related data.

```
student = {  
    "name": "John Doe",  
    "student_id": 1234567  
}  
  
another_student = {  
    "name": "Jane Smith",  
    "student_id": 7654321  
}
```

```
class Student:  
    def __init__(self, name, student_id):  
        self.name = name  
        self.student_id = student_id  
  
# Creating objects for students  
student_1 = Student("John Doe", 1234567)  
student_2 = Student("Jane Smith", 7654321)  
  
# Accessing attributes  
print(student_1.name, student_1.student_id)  
print(student_2.name, student_2.student_id)
```

User Defined Data Types (1)

Data Structure is not a data type!

- Questions:
 - Because the basic data types (e.g., char, int) are not enough, can we define data types we want to use by ourselves?
- Answer:
 - In addition to the basic data types, we can consider any **user-defined data structure**, like the student example in the previous slide, as a user-defined data type by specifying a set of operations associated with the user-defined data structure.

User Defined Data Types (2)

- Question/Answer:
 - Should the basic data types and the user-defined data types be treated in the same way?
 - The answer is yes. But, in reality, it depends on programming languages.
- For a basic data type, for example integer, we must use the operations provided by a programming language.
 - We do not need to know how integers are represented in the main memory.
 - We do not need to know how the integer operations are implemented.
- For a user-defined data type, we know it very well!
 - We know **how it is represented** in the main memory.
 - We also know **how they are implemented**.

User Defined Data Types (3)

- What are wrong if we know very well about the data structure?!
- Answer:
 - We may write programs depending on our knowledge on the data representations and data manipulations heavily.
 - We cannot change a user-defined data type (its representation and/or its implementations) easily when needed. This is because we can not easily figure out how others use this data type.
 - Example:
 - `student_1 = Student("John Doe", 1234567)`
 - `student_1.student_id = 7654321`
- Action:
 - We need to **hide details**! So developers cannot possibly know too much. (Very Important)

User Defined Data Types (4)

- How to hide details?!
- Answer:
 - Use a class to define a user-defined data type.
 - Use methods to manipulate the data and hide implementation details.
 - Avoid direct access to attributes by marking them private (using `_` or `__`).

```
class Student:
    def __init__(self, name, student_id):
        self.__name = name          # Private attribute
        self.__student_id = student_id # Private attribute

    # Public method to access private data
    def get_student_id(self):
        return self.__student_id

    # Public method to update private data
    def set_student_id(self, new_id):
        if isinstance(new_id, int) and new_id > 0:
            self.__student_id = new_id
        else:
            raise ValueError("Invalid student ID!")
```

Abstract Data Types (ADT)

- An ADT defines what operations can be performed, but hides how they are implemented.
- It separates the interface (what the ADT does) from the implementation (how it works).

An ADT Example of Set

- Data values: {1, 3, 5, 8}, {5, 8, 12}, ...
- Operations:
 - `search(integer, set)`: search if an integer is in the set
 - `intersection(set, set)`: return the intersection of two sets
 - `union(set, set)`: return the union of two sets
- The details
 - The representation: arrays or lists or ...
 - The implementation of operations: it can be implemented in many different ways depending on the data representation and the programming language used.

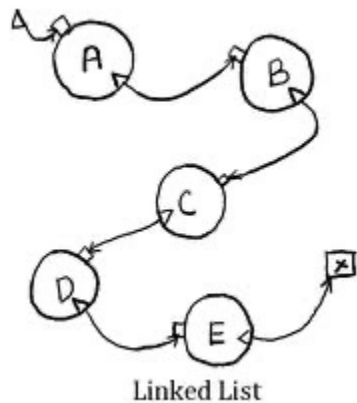
What Are the Common ADTs?

- Lists, stacks, queues, trees, graphs, etc.
 - Group same objects together
- The efficiency of ADTs
 - The data representation: data structures
 - The implementation of the operations: algorithms

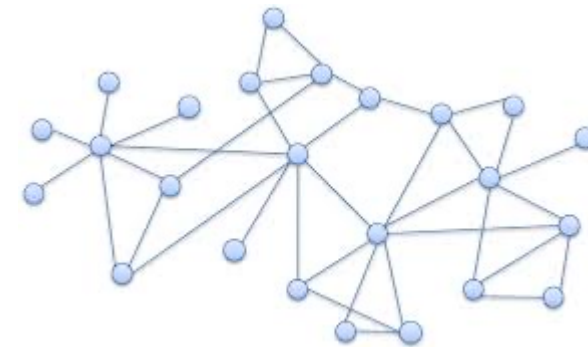
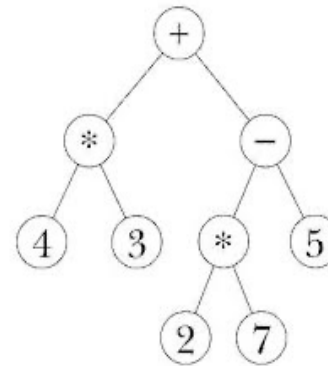
Array:

23	4	6	15	5	7
0	1	2	3	4	5

↑
Array index



Tree



A Summary on Data Types

- Data Structure (Representation/Organization)+Operations = Data Type
- Data Types
 - Basic Data Type provided by a programming Language
 - User Defined Data Type by programmers when needed
- A User Defined Data Type
 - Can be built on other data types (either basic or user-defined).
- Abstract Data Types
 - Two Separations:
 - The definition of the data values is separated from their representation.
 - The definition of the operations is separated from their implementation.
 - Encapsulation: Hides internal details of implementation.

Programming Language Support for ADTs

- Object-Oriented Programming Languages:
 - E.g. Python, C++, Java (via the concept of class)
- C Programming Language:
 - C does not have an explicit mechanism for implementing ADTs.
 - However, it is still possible and desirable to design ADTs in C

Array-Based Sequences

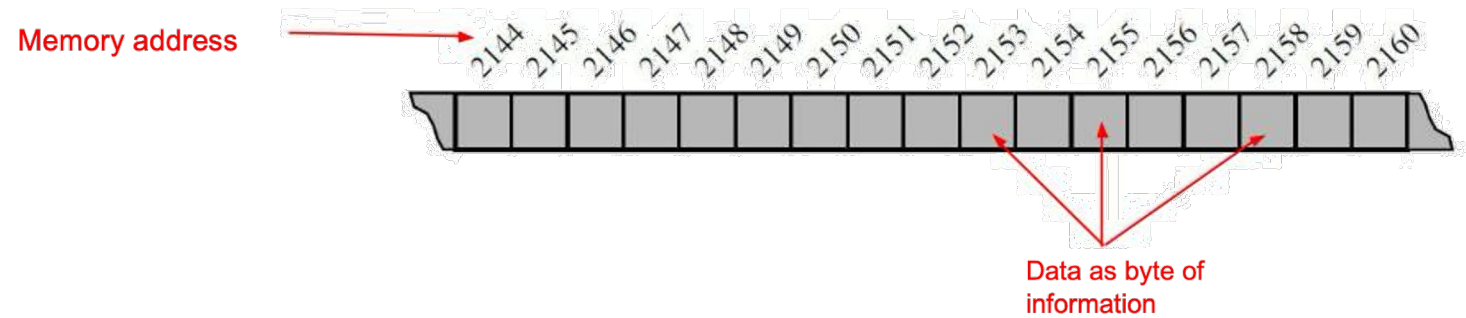
Outline

- Python Sequence Types
- Low-level arrays
- Dynamic Arrays
- Multidimensional Arrays

Python's Sequence Types

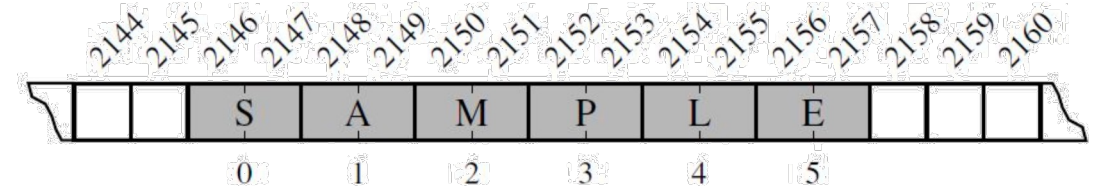
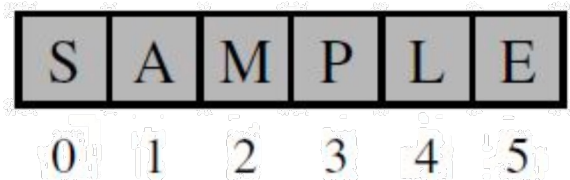
- Sequence classes: **list**, **tuple** and **str**
- Common behaviour:
 - Each supports indexing to access an individual element of a sequence: e.g., `seq[k]`.
 - Each uses a low-level concept known as an **array** to represent the sequence.
- Difference lies in:
 - The abstraction these classes represent and how these complex data structures are implemented.
- A deeper understanding of these data structures is important for a good programmer.

Low-level Arrays



- The primary memory of a computer is composed of bits of information.
- Those bits are typically grouped into larger units that depend upon the precise system architecture. Such a typical unit is a **byte**, which is equivalent to 8 bits.
- Each byte of information in the memory is associated with a unique number that serves as its address – **memory address**.
- **Random Access Memory (RAM)**
 - Despite the sequential layout of memory locations, computer hardware is designed to access any memory location efficiently using its memory address. That is, it is just as easy to retrieve byte #8675309 as it is to retrieve byte #309.
- Any individual byte of memory can be stored or retrieved in $O(1)$ time.

- An **array** is a group of related variables stored one after the another in a contiguous portion of the computer's memory.
- Example: A text string is stored as an ordered sequence of individual characters.
- Each cell of an array must use the same number of bytes. This allows an arbitrary cell to be accessed in constant time.



- Each unicode character is represented by 16 bits or 2 bytes.
- A six-character string such as 'SAMPLE' would be stored in 12 consecutive bytes of memory.
- This is an array of 6 characters and requires 12 bytes of memory.
- Each location within the array is called a **cell** and an integer **index** will be used to describe the cell location in the array.
- As example, cell with index 4 contains 'L' that is stored in memory locations 2154 and 2155 respectively.
- Address of any cell with index $k = \text{start} + \text{cellsize} * k$.

Dynamic Arrays

- When creating low-level arrays, it is necessary to specify the precise size of that array explicitly during instantiation.
- The same is applicable to Python's **tuple** and **str** instance which are immutable - Once created, its size or content cannot be altered.
- Python's **list** class represents a **dynamic array** where elements could be added to or removed from the list.
- Semantics of Dynamic Array
 - It maintains an underlying array that has greater capacity than the current length of the list.
 - If the reserved capacity is exhausted, the system initializes a new array larger in size and the older array is then reclaimed by the system.

- An empty list requires some space (64 bytes on this machine). It is required for keeping the following information:
 - Number of actual elements currently stored in the list.
 - The maximum number of elements that could be stored.
 - The reference to the currently allocated array.
- Each memory address is a 64-bit (8 byte) number for a 64-bit CPU PC.
- Reserve capacity increases 32 bytes (4 cells) to 64 (8 cells) and so on as the array is extended continuously by adding new objects.

```

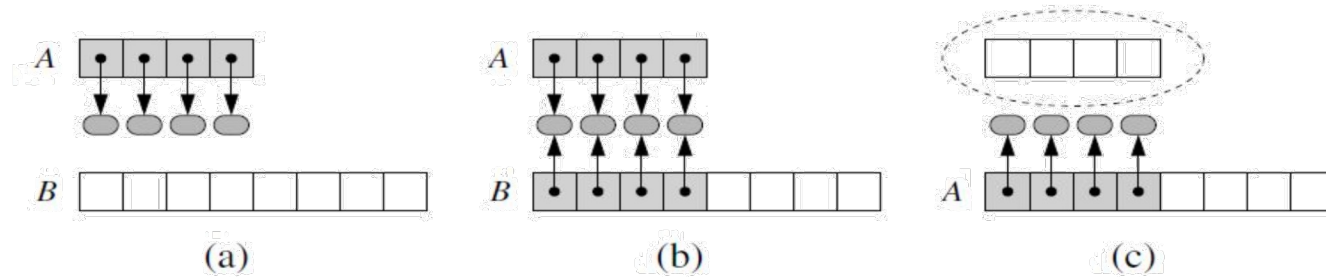
1 # analyzing a dynamic array
2 import sys
3 data = []
4 for k in range(27):
5     a = len(data)
6     b = sys.getsizeof(data)
7     print('Length: {0:3d}; Size in bytes: {1:4d}'.format(a,b))
8     data.append(None)

```

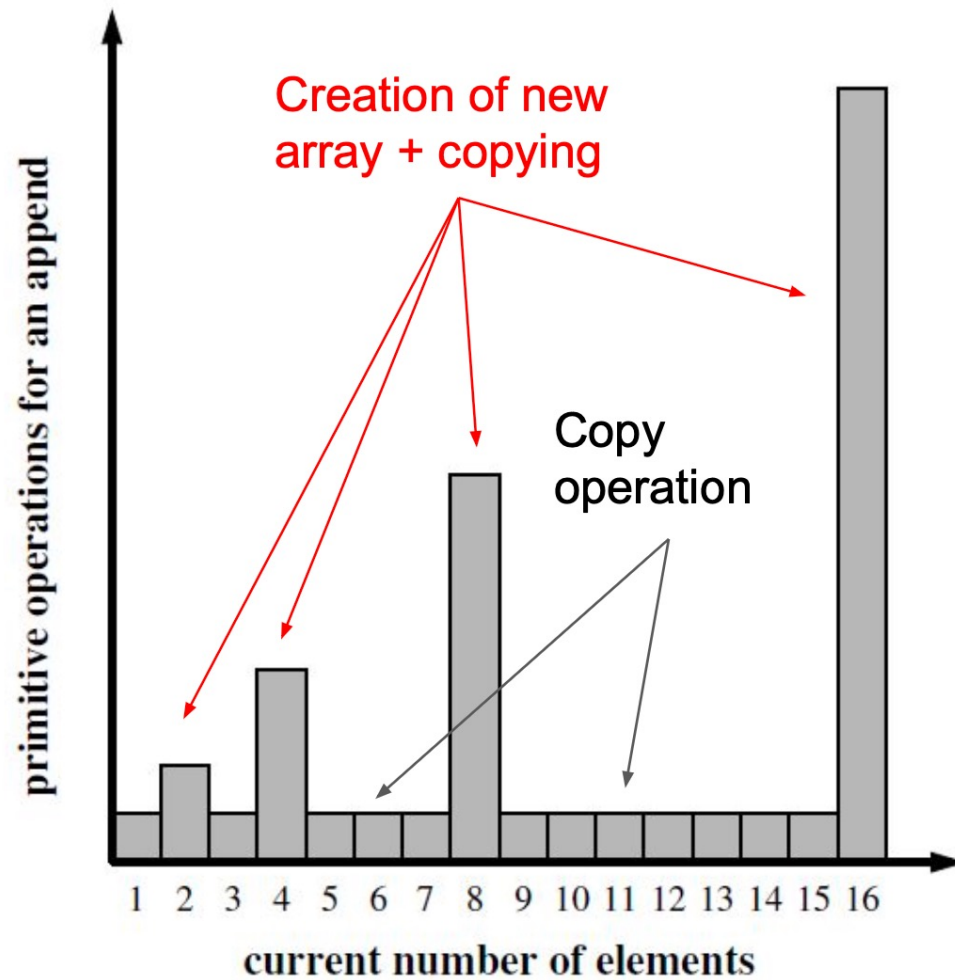
Length: 0; Size in bytes: 64	
Length: 1; Size in bytes: 96	
Length: 2; Size in bytes: 96	
Length: 3; Size in bytes: 96	
Length: 4; Size in bytes: 96	32 bytes are reserved for storing 4 object references.
Length: 5; Size in bytes: 128	
Length: 6; Size in bytes: 128	
Length: 7; Size in bytes: 128	
Length: 8; Size in bytes: 128	Space for 4 object references
Length: 9; Size in bytes: 192	
Length: 10; Size in bytes: 192	
Length: 11; Size in bytes: 192	
Length: 12; Size in bytes: 192	
Length: 13; Size in bytes: 192	
Length: 14; Size in bytes: 192	
Length: 15; Size in bytes: 192	
Length: 16; Size in bytes: 192	Space for 8 object references stored.
Length: 17; Size in bytes: 264	
Length: 18; Size in bytes: 264	
Length: 19; Size in bytes: 264	
Length: 20; Size in bytes: 264	
Length: 21; Size in bytes: 264	
Length: 22; Size in bytes: 264	
Length: 23; Size in bytes: 264	
Length: 24; Size in bytes: 264	
Length: 25; Size in bytes: 264	
Length: 26; Size in bytes: 344	Space for 9 object references stored.

Implementing a Dynamic Array

- The key is to provide a means to grow the array A that stores the elements of a list.
- If an element is appended to a list at a time when the underlying array is full, we perform the following steps:
 - Allocate a new array B with larger capacity.
 - Set $B[i] = A[i]$, for $i = 0, \dots, n - 1$ where n denotes the current number of items.
 - Set $A = B$, i.e., we use B as the array supporting the list.
 - Insert the new element in the new array.



- How large of a new array to create?
 - Common rule - the new array to have twice the capacity of the existing array that is filled.



Run-time of a series of append operations on a dynamic array

```

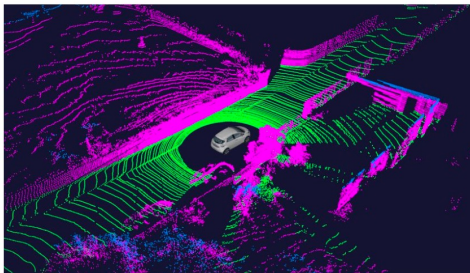
1 # Implementing a dynamic array
2 import ctypes
3 class DynamicArray:
4     ''' A dynamic array class akin to a simplified Python list '''
5     def __init__(self):
6         ''' create an empty array '''
7         self._n = 0                # count actual elements
8         self._capacity = 1         # default array capacity
9         self._A = self._make_array(self._capacity) # low-level array
10
11     def __len__(self):
12         ''' return number of elements stored in the array'''
13         return self._n
14
15     def __getitem__(self, k):
16         ''' return element at index k '''
17         if not 0 <= k < self._n:
18             raise IndexError('invalid index')
19         return self._A[k]
20
21     def append(self, obj):
22         ''' Add object to the end of the array'''
23         if self._n == self._capacity: # no enough room
24             self._resize(2*self._capacity) # so double the capacity
25         self._A[self._n] = obj
26         self._n += 1
27
28     def _resize(self, c):
29         ''' Resize the internal array to capacity c'''
30         B = self._make_array(c) # new (bigger) array
31         for k in range(self._n):
32             B[k] = self._A[k] # copy data from old array into new array
33         self._A = B # use the bigger array
34         self._capacity = c
35
36     def _make_array(self, c): # nonpublic utility
37         ''' Return new array with capacity c'''
38         return (c * ctypes.py_object)()
39
40 A = DynamicArray()
41 A.append(1)
42 A.append(2)
43 A.append(7)
44 print(A[0], A[1], A[2])

```

1 2 7

Multi-Dimensional Data Sets

- Have more than one dimensions. A two-dimensional list may be called a matrix.
- Examples:
 - Geographical information may be naturally represented in two-dimensions.
 - Range data obtained from a lidar is a 3D data.
 - Monochrome images are 2D images.



	0	1	2	3	4	5	6	7	8	9
0	22	18	709	5	33	10	4	56	82	440
1	45	32	830	120	750	660	13	77	20	105
2	4	880	45	66	61	28	650	7	510	67
3	940	12	36	3	20	100	306	590	0	500
4	50	65	42	49	88	25	70	126	83	288
5	398	233	5	83	59	232	49	8	365	90
6	33	58	632	87	94	5	59	204	120	829
7	62	394	3	4	102	140	183	390	16	26

Two-dimensional integer dataset (Number of stores in various regions in district)

```
22 18 709 5 33
45 32 830 120 750
4 880 45 66 61
```

```
Data = [ [22,18, 709, 5, 33], [45, 32, 830, 120, 750],
          [4, 880, 45, 66, 61] ]
```

Each element can be accessed by using a syntax:
`data[1][3]`

Constructing Multi-Dimensional List

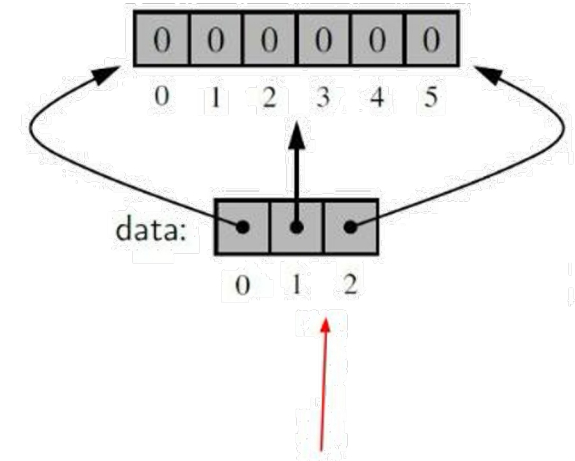
- Quickly initialize a one-dimensional list:

`Data = [0] * n`

- Initialize multi-dimensional list:

`Data = ( [0] * c ) * r`

`Data = [ [0] * c ] * r`



Warning: this is a mistake

Warning: still a mistake

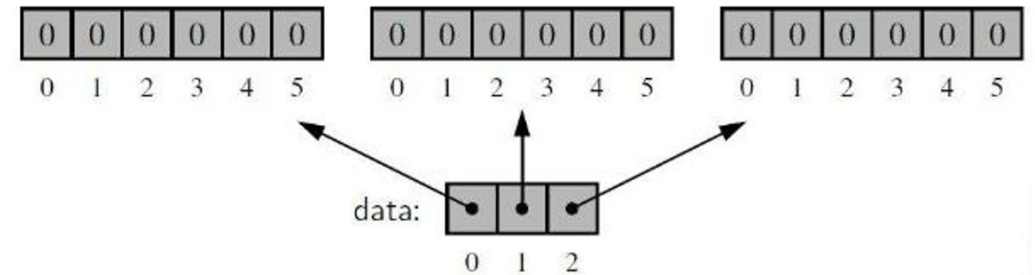
```
1 data = [[0]*3]*3
2 print(data)
3 data[2][0] = 5
4 print(data)
```

```
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
[[5, 0, 0], [5, 0, 0], [5, 0, 0]]
```

Correct Implementation for Initializing a 2-Dimensional List

`data = [[0] * c for j in range(r)]`

- This ensures that each cell of the primary list refers to an independent instance of a secondary list.



```
1 # data = [[0]*3]*3
2 # print(data)
3 # data[2][0] = 5
4 # print(data)
5
6 data2 = [[0]*4 for j in range(3)]
7 print(data2)
8
9 data2[2][0] = 5
10 data2[0][1] = 2
11 print(data2)
```

```
[[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
[[0, 2, 0, 0], [0, 0, 0, 0], [5, 0, 0, 0]]
```

```

1 # Tic-Tac-Toe Game
2 class TicTacToe:
3     '''
4     Management of a Tic-Tac-Toe Game
5     No strategy
6     '''
7     def __init__(self):
8         ''' start a new game'''
9         self._board = [ [' ' ] * 3 for j in range(3) ]
10        self._player = 'X'
11
12    def mark(self, i, j):
13        ''' Put X or O mark at position (i,j) for next player's turn '''
14
15        if not(0 <= i <= 2 and 0 <= j <= 2):
16            raise ValueError('Invalid board position')
17        if self._board[i][j] != ' ':
18            raise ValueError('Board position occupied')
19        if self.winner() is not None:
20            raise ValueError('Game is already complete')
21        self._board[i][j] = self._player
22
23        if self._player == 'X':
24            self._player = 'O'
25        else:
26            self._player = 'X'
27
28    def _is_win(self, mark):
29        ''' Check whether the board configuration is a win for a given player'''
30        board = self._board
31
32        return (mark == board[0][0] == board[0][1] == board[0][2] or # row 0
33                mark == board[1][0] == board[1][1] == board[1][2] or # row 1
34                mark == board[2][0] == board[2][1] == board[2][2] or # row 2
35                mark == board[0][0] == board[1][0] == board[2][0] or # column 0
36                mark == board[0][1] == board[1][1] == board[2][1] or # column 1
37                mark == board[0][2] == board[1][2] == board[2][2] or # column 2
38                mark == board[0][0] == board[1][1] == board[2][2] or # diagonal
39                mark == board[0][2] == board[1][1] == board[2][0]) # rev diag
40
41    def winner(self):
42        ''' return mark of the winning player, or None to indicate a tie'''
43        for mark in 'XO':
44            if self._is_win(mark):
45                return mark
46        return None
47
48    def __str__(self):
49        ''' return th estring representation of the current game board'''
50        rows = ['|'.join(self._board[r]) for r in range(3)]
51        return '\n-----\n'.join(rows)
52

```

```

53
54 game = TicTacToe()
55
56 # X moves          # O moves
57 game.mark(1,1);    game.mark(0,2)
58 game.mark(2,2);    game.mark(0,0)
59 game.mark(0,1);    game.mark(2,1)
60 game.mark(1,2);    game.mark(1,0)
61 game.mark(2,0);
62
63 print(game)
64 winner = game.winner()
65 if winner is None:
66     print('Tie')
67 else:
68     print(winner, 'wins')
69

```

```

O|X|O
-----
O|X|X
-----
X|O|X
Tie

```

In class Exercise: Generalized Win Check

- Rewrite the `_is_win(mark)` function **without hard-coded indices**, using **loops over a 2-D array**.
- The solution should:
 - Check **all rows**
 - Check **all columns**
 - Check **both diagonals**
- Extension (optional)
 - Modify the function so it works for **any board size N**

Summary

- We studied the following in this lecture:
 - Working of the low-level arrays - initialization, accessing elements etc.
 - Creating and managing dynamic array
 - Creating multi-dimensional arrays